S01 — CPU vs GPU Performance

A Comparative Study of OpenMP and CUDA Matrix Multiplication

Abstract

This case study compares multicore CPU execution and GPU execution for square matrix multiplication across six problem sizes, from 128 × 128 to 3072 × 3072. The CPU implementation uses C++ and OpenMP with four reported CPU threads. The GPU implementation uses CUDA C++ and a tiled kernel that uses shared memory. The supplied measurements include CPU execution time, host-to-device transfer time, kernel execution time, device-to-host transfer time, GPU component-sum time, speedup, estimated GPU throughput, and transfer fraction. In the recorded run, GPU execution was faster at every tested size, with the measured speedup increasing from 1.306× at N = 128 to 30.592× at N = 3072. The transfer fraction generally decreased at larger sizes, indicating that computation accounted for a larger share of the measured GPU component time. These findings are descriptive of the supplied run; repeated trials and further controls are needed before treating them as statistically robust or generalizing them to all CPU/GPU workloads.

1. Introduction

CPUs and GPUs execute computations using different parallel-processing designs. A multicore CPU can distribute work among CPU threads, while a GPU can execute many lightweight threads concurrently. GPU acceleration is not automatically beneficial for every task: small workloads may be dominated by launch and transfer overhead, whereas larger workloads may provide enough parallel computation to offset that overhead.

This study uses matrix multiplication as a compute-intensive benchmark. The experiment varies the matrix dimension and compares the time required by an OpenMP CPU implementation with the time required by a CUDA implementation, including measured data transfers.

2. Research Question and Objectives

Research question: For which matrix sizes does GPU execution provide a meaningful performance benefit over multicore CPU execution when data-transfer time is included?

- Vary the square matrix dimension and observe how runtime changes.
- Measure CPU and GPU execution separately.
- Measure host-to-device and device-to-host transfer durations.
- Calculate speedup, GPU throughput, and transfer fraction.
- Interpret execution-time, speedup, and transfer-fraction graphs.

3. Experimental Design and Implementation

3.1 Benchmark workload

The benchmark computes C = A × B for two N × N matrices. The operation count is approximated as $2N^3$ floating-point operations. The supplied programs initialize A with 1.0 and B with 2.0, so each output element should equal 2N. The check values in the results (256, 512, 1024, 2048, 4096, and 6144) match this expected value for the six matrix sizes.

3.2 CPU implementation

The CPU implementation is written in C++ and uses OpenMP to distribute the outer matrix-row loop across threads. The recorded output reports four CPU threads for all six sizes. The CPU timing is measured around the matrix multiplication loop, excluding vector allocation and initialization.

3.3 GPU implementation

The GPU implementation is written in CUDA C++. A tiled kernel uses shared-memory tiles of 16 × 16 elements to reuse matrix values and reduce repeated global-memory access. The recorded GPU timings separate host-to-device (H2D) transfer, kernel execution, and device-to-host (D2H) transfer. The reported GPU component-sum time is the sum of those three measured components.

3.4 Platform and timing scope

The experiment was run in a Kaggle notebook with an NVIDIA GPU and a Linux/CUDA toolchain. The submitted results do not identify the exact GPU model, CPU model, compiler versions, or clock state, so these details should be added from the notebook environment if required by the course. The results file contains one recorded measurement per size and does not document warm-up counts or repeated trials.

4. Metrics and Formulas

Metric	Formula used	Interpretation
CPU execution time	Measured CPU loop time (ms)	Time taken by OpenMP matrix multiplication.
GPU component-sum time	H2D + kernel + D2H (ms)	Sum of the three reported GPU timing components.
Speedup	CPU time / GPU component-sum time	Values above 1 indicate a faster measured GPU component sum.
GPU throughput	$2N^3$ / GPU time, converted to GFLOP/s	Approximate floating-point work completed per second.
Transfer fraction	(H2D + D2H) / GPU component-sum × 100%	Share of the GPU component sum attributed to transfers.

Important timing qualification: the supplied speedup and throughput values use the GPU component-sum column, not an independently measured end-to-end wall-clock duration around the entire GPU operation. This is the correct interpretation of the values shown in the submitted results, but it may omit host-side launch and other overheads. Therefore, the report calls these values component-sum-based measurements rather than claiming they are a complete application-level end-to-end benchmark.

5. Results

Table 1 reproduces the key values visible in the supplied results PDF. Times are in milliseconds, throughput is in GFLOP/s, and transfer fraction is a percentage.

N	CPU threads	CPU (ms)	H2D (ms)	Kernel (ms)	D2H (ms)	GPU sum (ms)	Speedup (×)	Throughput (GFLOP/s)	Transfer (%)
128	4	0.533	0.087	0.279	0.042	0.408	1.306	10.276	31.611
256	4	2.457	0.180	0.093	0.094	0.368	6.680	91.244	74.643
512	4	20.024	0.746	0.561	0.365	1.671	11.981	160.609	66.452
1024	4	158.058	2.113	4.293	1.023	7.429	21.277	289.084	42.212
2048	4	1262.860	7.663	34.318	3.983	45.964	27.475	373.771	25.337
3072	4	4645.470	17.722	124.809	9.321	151.852	30.592	381.833	17.809

Table 1. Measurements transcribed from the submitted results PDF.

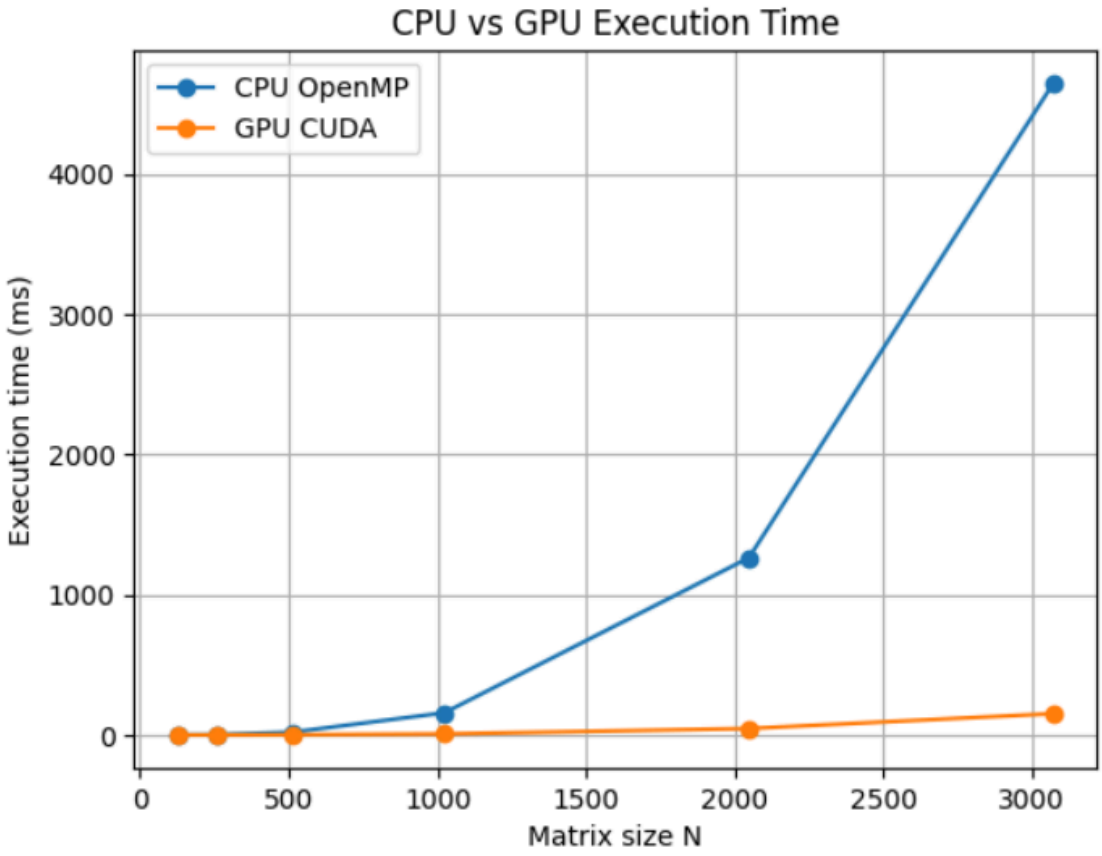
6. Graphical Analysis

6.1 Execution time vs matrix size

The CPU execution time rises from 0.533 ms at N = 128 to 4645.470 ms at N = 3072. The GPU component sum rises from 0.408 ms to 151.852 ms over the same range. The gap between the curves becomes much larger at

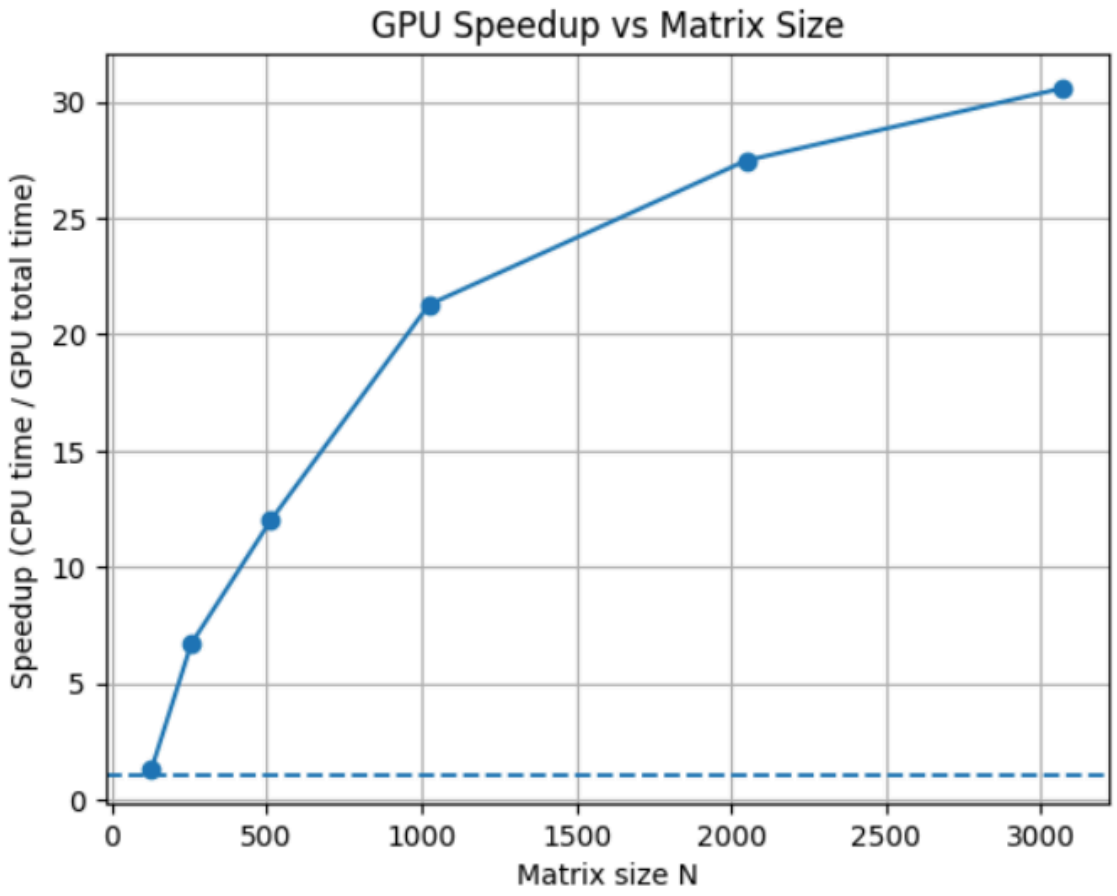
high matrix sizes. This is consistent with the GPU implementation exploiting parallelism more effectively for the larger compute-intensive matrices in this experiment.

	size	cpu_threads	cpu_ms	cpu_check	h2d_ms	kernel_ms	d2h_ms	gpu_component_sum_ms	gpu_check	gpu_ms	speedup	throughput_gflops	transfer_fraction_pct
0	128	4	0.533	256	0.087	0.279	0.042	0.408	256	0.408	1.306	10.276	31.611
1	256	4	2.457	512	0.180	0.093	0.094	0.368	512	0.368	6.680	91.244	74.643
2	512	4	20.024	1024	0.746	0.561	0.365	1.671	1024	1.671	11.981	160.609	66.452
3	1024	4	158.058	2048	2.113	4.293	1.023	7.429	2048	7.429	21.277	289.084	42.212
4	2048	4	1262.860	4096	7.663	34.318	3.983	45.964	4096	45.964	27.475	373.771	25.337
5	3072	4	4645.470	6144	17.722	124.809	9.321	151.852	6144	151.852	30.592	381.833	17.809



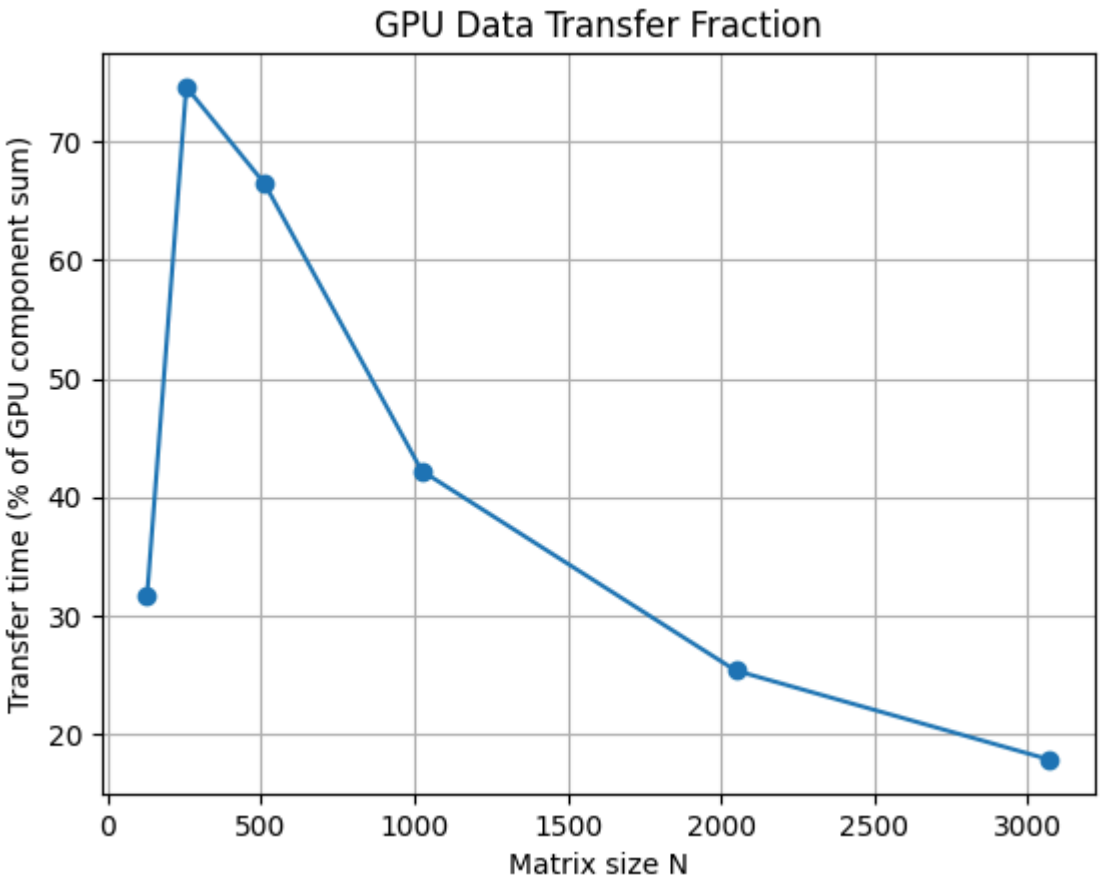
6.2 Speedup vs matrix size

The component-sum-based speedup increases at every tested size: 1.306×, 6.680×, 11.981×, 21.277×, 27.475×, and 30.592×. At N = 128 the observed advantage is modest. At N = 3072, the recorded CPU time is approximately 30.6 times the GPU component sum. Because only six sizes were tested, the exact crossover point between CPU and GPU is not resolved; the GPU is already faster at the smallest tested size in this run.



6.3 GPU transfer fraction

The transfer fraction is 31.611% at N = 128, rises to 74.643% at N = 256, and then declines across the larger tested sizes to 17.809% at N = 3072. The pattern is not monotonic at the two smallest sizes, which may reflect the very short measured durations and timing variability. For the largest matrices, the kernel accounts for a larger share of the component sum, so the measured transfer share is smaller.



7. Discussion

- Problem size matters: the observed speedup grows as N increases, supporting the idea that larger matrix multiplications provide more parallel work to amortize GPU overhead.
- Transfer overhead is meaningful: transfers account for a substantial fraction of total measured GPU component time at several sizes, particularly N = 256 in this run.
- Kernel time is not the whole story: judging GPU performance from kernel time alone would understate the cost of moving data between host and device.
- The CPU baseline is specific to this implementation: the CPU uses a straightforward loop ordering, while the CUDA kernel uses shared-memory tiling. The comparison therefore reflects both hardware and implementation choices.
- The results are encouraging but preliminary: a single recorded measurement per size cannot quantify variability or establish statistical significance.

8. Limitations and Improvements

- Repeat each size at least five times after warm-up and report median, minimum/maximum, and standard deviation or interquartile range.
- Measure a host wall-clock end-to-end GPU duration around H2D transfer, kernel execution, and D2H transfer, with appropriate synchronization; use that duration for primary speedup and throughput.
- Record the exact GPU model, CPU details, CUDA and C++ compiler versions, and the number of available CPU cores.
- Run CPU tests with multiple thread counts (for example, 1, 2, and 4) where the environment permits, and record the actual thread count used.

- Use seeded random matrices and compare the full CPU and GPU output arrays with a floating-point tolerance. The current constant-input check confirms a simple expected output but is not a comprehensive numerical validation.
- Consider an optimized CPU matrix multiplication or a BLAS library as an additional baseline, while retaining the current OpenMP implementation for reproducibility.

9. Conclusion

For the six matrix sizes in the supplied Kaggle run, the CUDA implementation had a lower reported component-sum time than the four-thread OpenMP implementation. The measured speedup increased from 1.306× at N = 128 to 30.592× at N = 3072. The transfer fraction decreased overall at larger sizes, reaching 17.809% for N = 3072, although it was high and non-monotonic among the two smallest sizes. These observations suggest that GPU execution offered its strongest benefit for the largest tested matrix multiplication. The conclusion is limited to this benchmark and these recorded measurements; repeated trials and end-to-end wall-clock timing are recommended before making broader claims about CPU-versus-GPU performance.

Appendix A. Key values for quick reference

- Smallest tested matrix: N = 128; speedup = 1.306×; transfer fraction = 31.611%.
- Largest tested matrix: N = 3072; CPU time = 4645.470 ms; GPU component sum = 151.852 ms; speedup = 30.592×; throughput = 381.833 GFLOP/s; transfer fraction = 17.809%.
- Maximum transfer fraction in the supplied run: 74.643% at N = 256.
- All reported correctness-check values equal 2N, as expected for the constant inputs A = 1 and B = 2.